Evidence-Based Agent Memory from Self-Reported Learning Cues

OpenBot

***Abstract*—An agent's report can locate work worth checking without giving a sound account of that work. We study how to use such cues for lasting memory. A structural baseline accepts source links but does not check claim support, scope, or current intent. We define an evidence contract with typed claims, a bounded evidence queue, a support graph, and independent audits of silent tasks. We prove a coverage bound, give an unbiased estimate of missed claims under stated sampling assumptions, and prove an eligibility invariant for atomic correction. Executable finite models expose loss of an early correction, unsupported claims with valid sources, and stale dependent observations. These results establish properties of the stated rules. They do not establish extraction accuracy, poisoning resistance, or later task gains. A controlled study design separates these outcomes and measures the cost of each mechanism.**

***Index Terms*—Agent memory, Evidence, Self-report, Intent preservation**

I. Introduction

Lasting memory carries useful decisions between tasks, but also carries errors. A system must decide when to inspect an event, which claims it supports, when those claims cease to apply, and how to use them without changing the next request.

An agent may report after a failed command, a correction, or an uncertain choice. The report can locate work worth examining. Yet it may credit the wrong repair, repeat an outside instruction, or turn a local exception into a general rule. Saving that account as a lesson can make its error persistent.

We study a separation of these steps. The agent emits a report event with a short account of its work. The event queues a learning job, but its text is excluded from the evidence. A separate helper reads the task record and proposes memory claims. Thus the working agent chooses an occasion for learning without directly choosing the lasting lesson. We define both a deliberately limited baseline and a proposed extension; neither is presented as a deployed system.

"Self-report" means generated text and a report event, without a claim about feelings or inner awareness. A prompt can invite reports of errors or uncertainty. Its effect must be measured as behavior. Omitting the memory function from that prompt does not prove model ignorance of it. We make no claim that concealed memory is better than disclosed memory.

The contribution is a testable contract for the boundaries between event selection, evidence, lasting claims, and present action. It combines a precise structural baseline, correction and queue invariants, a sampling estimate for silent work, and executable counterexamples. Dependency tracking and weighted sampling are established methods. We apply them to separate failure modes that a single memory accuracy score hides. The reported experiments are finite rule models. No language-model memory benchmark or user study is reported.

The research variable is the truth and later value of memory under a chosen trigger policy. We assume entries have passed access checks for the current reader. Cross-user information flow is a separate concern. We hold the model context limit fixed; general context compression is also outside this study.

II. Prior work and the comparison space

Reflexion uses trial feedback to produce verbal reflections kept for later trials, without changing model weights. Its separation of actor, evaluator, and reflection is relevant here: feedback can support a lesson, but an explanation is a further inference. In our defined pipeline, a cue may precede judgment of the outcome; the learner reads recorded items rather than accepting the cue as the lesson. This changes the claim's source, not the need for sound feedback. [1]

Generative Agents forms reflections from stored experience. Its implementation starts reflection when recent importance scores sum past a threshold. Both that method and cue-driven learning select moments for more expensive synthesis, but use different signals. Its published objective includes believable simulated behavior. That does not establish accurate memory for an agent carrying out user requests. [2]

MemoryBank combines retrieval, updates, and forgetting based on time and importance. Our concern differs from decay: recent unsupported claims can be harmful, while old valid procedures can be useful. Forgetting and invalidation could work together, but age alone cannot distinguish a correction from an unrelated later event. [3]

MemGPT gives agents functions to move information between stores and alter working context. A direct writer chooses both the occasion and content of a write. Our proposed pipeline assigns content selection to a helper. The evaluation holds storage capacity and prompt allowance constant between these choices. Memory paging is outside this paper's scope. [4]

LoCoMo evaluates long conversations through question answering, event summaries, and dialogue generation. Its conversations are generated and then checked and edited by people. LongMemEval tests information extraction, reasoning across sessions, time reasoning, updates, and abstention. These works motivate testing both continuity and correction over time. Neither a single correct recall answer nor a fluent conversation establishes that memory kept the user's current task intact. Our proposed protocol adds action-level intent checks and direct labels for admitted claims. [5], [6]

AgentPoison studies attacks on memory and retrieval stores, where selected material changes later agent behavior. Its setup differs from a chat-based evidence learner, so its success rates do not predict outcomes for the proposed design. It motivates testing persistence of harmful claims after outside content enters a valid task. [7]

ALCE evaluates generated answers with separate measures for correctness and citation quality. It establishes a close comparison for source checking: a present citation and a supported assertion are different tests. We extend that evaluation boundary over time, to corrections, dependent claims, and later actions. We do not claim that the source-support distinction is new. [8]

Doyle's truth maintenance system records reasons for beliefs and revises beliefs when their assumptions change. Our support graph uses this established idea. We restrict it to immutable, acyclic dependencies whose listed premises are all required. The added question is how a fallible text model supplies such premises and how a corrected graph becomes visible to a later reader. Graph repair cannot make an unsound inference sound. [9]

Horvitz and Thompson estimate finite population totals using known inclusion probabilities. We use this method for claims missed by cue-only learning. It measures a blind spot in the trigger policy; it does not estimate model truthfulness from self-reports. The unit, inclusion rule, and complete labeling assumption must be fixed before the estimate has this meaning. [10]

TABLE I
DEFINED COMPARISON POLICIES. ALL CAN SHARE STORAGE AND A READER; THE FINAL ROW ADDS THE PROPOSED CHECKS.

Policy	**When it learns**	**Basis of a write**
Direct writing	Agent chooses a write	Agent's proposed memory
Periodic reflection	Time, count, or importance rule	A summary or inferred lesson
Cue and extraction	Agent reports an event	A helper's claim with source links
Proposed mixed policy	Cue plus audit of silent work	Checked claim and source span

III. PROBLEM MODEL

Let a chat history be an ordered set of items. Each item has an identifier, a turn, a role, and content. A role records whether text came from the user, the assistant, or a tool. The learner receives an evidence window drawn from this history. A cue is an event that makes such a window eligible for learning. Cue content and evidence content are distinct objects.

A memory entry contains a claim, layer, source links, tags, status, and times. A fact points to chat items. An observation infers from other entries. A pinned entry records a person's explicit request to remember something. An observation's confidence is model output, not a measured probability unless a calibration study supports that meaning.

We separate four properties. Source validity asks whether cited records exist in the allowed evidence. Support asks whether those records justify the stated claim at its stated scope. Current validity asks whether later evidence has displaced it. Usefulness asks whether adding it helps a later task. These properties need not agree. A correct statement about last month's deadline may be useless today. A useful guess may lack support. A claim that the user stated a deadline can be supported even when the deadline is wrong in the outside world.

The proposed contract therefore treats facts as supported assertions, with attribution and time when needed. It does not promise a universal truth store. User preferences and instructions require careful scope: "use plain text for this report" is narrower than "always use plain text." A claim about a tool outcome should say which run and checks support it. A claim about a person should avoid turning one action into a lasting trait.

We assume stable item identifiers and an ordered record of new evidence. We assume that the model may make mistakes in extraction, generalization, source selection, and rewriting. Outside tool content may contain instructions chosen by an attacker. User input can also be incomplete or mistaken. The store and its reference checks are trusted to implement their stated rules; the prose supplied to a model is not trusted merely because it has an identifier.

A. Evidence, state, and failure boundaries

An evidence item is an immutable text snapshot with its role, task, order, and content digest. A digest checks that the bytes stayed the same; it supplies no evidence that those bytes are true. An edit creates a new item that names the earlier item it replaces. A source span identifies a snapshot and a byte range, with the context used for review. The learner cannot silently exchange new text under an old source identifier.

A proposed claim record contains a proposition, subject, scope, valid-time interval, source spans, support kind, required premises, and review state. Valid time says when the claim applies; record time says when the store learned it. A later correction can change the former without rewriting the latter. The support kinds are **reported**, **observed**, and **inferred**. "The user said the build passed" is reported evidence. "This build passed these checks" needs the recorded checks and outcomes. "This repair caused the pass" is an inference and needs further assumptions. The store must not convert one kind into another because the same words appear in several records.

The store states are pending, eligible, blocked, and retired. A model proposes a claim while it is pending. A separate support decision can make it eligible. Every change from eligible to blocked or retired also blocks its required descendants in the same commit. Retirement removes an entry from future use without deleting its audit record. A pinned instruction has the same scope and time fields; being pinned does not establish an outside fact or override a newer current instruction.

Workers can crash, retry a model call, or overlap. The database must serialize commits on a chat and its support

graph. Each proposal carries the source digests, premise revisions, cursor, and cue generation that it read. Before commit, the store checks those versions and current premise eligibility. A changed version discards the proposal for a fresh read. A model timeout makes no state change. A committed job may be read again after a lost reply; a unique batch key prevents a second write. These rules make storage transitions repeatable. They do not make model calls occur exactly once.

Corrections and dependent blocking have one commit point. Readers use a single committed view of entries and their required premises. A reader that took its view before a correction may still act on old memory. The claim here is that a read ordered after the correction sees the blocked state. Stopping an action already issued requires a separate action control rule.

IV. Defined structural baseline

A. State and trigger rules

The baseline state is an append-only history H, a pool M, and a cursor c, persistent generation counter q, and pending marker per chat. Items have unique increasing sequence numbers, a role, text, and an outside-data flag. That flag is false for user and assistant items; it is true for tool results unless the tool is an explicitly trusted local instrument. System notices and report arguments and replies are ineligible as evidence. Initially counters are zero and pools are empty.

A report increments q and marks its chat pending. Its wording has no other effect on learning. At a turn boundary with no waiting user input, a pending job takes a snapshot ending at sequence h. It collects eligible items with $c <$ sequence $\leq h$, then gives the last b items to an extraction model, in order. We define $b = 60$ for the fixtures. This chosen limit is a baseline parameter, not a measured optimum. The model receives the active pool and evidence, and returns an ordered plan. Without a report, this baseline performs no learning.

B. Exact admission rules

A fact proposal supplies a fresh identifier, nonempty claim, and nonempty list of evidence identifiers. Accept it if every reference is in the batch and at least one referenced item has the outside flag false. No entailment check occurs. Unknown references or outside-only support cause skipping. Accepted additions are active and receive creation and update times. These rules allow assistant text and mixed user/tool source lists.

An observation proposal supplies a fresh identifier, nonempty claim, nonempty support list, and finite confidence. Accept it if every named entry is active at that point in plan execution. Earlier accepted proposals in the same plan may supply support; later proposals may not. Clip confidence to the interval from zero to one. Confidence has no effect on admission. Source links cannot change after creation. Fresh identifiers and this ordering prevent cycles when the initial pool is empty.

An invalidation proposal names an active learned entry and a nonempty reason. Mark only that entry invalid, record the time and reason, and leave dependent entries unchanged. No reference to correcting evidence is required. Pinned entries, entered directly by a person, cannot be changed by the learner. Retracted entries are also protected. Topic labels carry no support meaning. Invalid proposals are skipped independently, so the remaining plan can still take effect.

Model work runs outside a store transaction. At commit, reread the pool, compare the saved cursor with c, and discard the result if another job moved it. Otherwise apply proposals in order against the latest pool, record the changes, and set the cursor to h, including when all proposals were skipped. Delete the pending marker only if its generation still equals the captured q. These state changes are atomic. On model failure or exhausted budget, keep the cursor and job unchanged. No claim about a real database follows from defining this transaction.

These rules check structure, not meaning. They allow unsupported claims, unreviewed dependents, and cursor movement past evidence never supplied to the learner.

C. Reading rule

For a new request x, supply the reader model with active eligible entries, ordered by decreasing update time and then identifier. Take the longest prefix that fits a fixed character allowance K, without splitting an entry. Do not check ancestors of an observation. The model proposes a full rewritten request y and a nonempty set S of used entry identifiers. Its prompt asks it to keep every request and constraint, add no tasks, and prefer x over conflicting memory.

The baseline uses y exactly when it is nonempty, differs from x, and every identifier in S belongs to the supplied prefix. Otherwise it uses x. Save x, y, and S for audit. There is no check of semantic equivalence, relevance, or whether the named entries justify the changes. This rule makes preservation of intent a prompt objective rather than an enforced property.

V. Proposed evidence contract

A. Treat cues as scheduling data

The proposed learner keeps cue text out of the claim evidence. It records the event for later analysis and uses it to schedule a read of the underlying task. A source span is a bounded piece of an item with enough surrounding text to interpret it. A cue may suggest where to start looking, but a candidate must still cite such spans. The same rule applies whether the tool is described as a reporting tool or explicitly as a learning tool.

We propose a mixed policy with two routes. A cue schedules inspection after the task reaches a stable boundary. A separately budgeted sample of silent tasks is also read. Before a run, draw and save one audit bit per task with a declared probability p, independently of agent reports. The algorithm queues a task when it has either a cue or an audit bit of one. Give all compared runs the same audit tape. This measures what cues miss and can capture routine facts that cause no report.

Coverage also needs a different cursor rule. Take the first b eligible records after the cursor, and advance only past records considered or explicitly discarded under a recorded rule. Keep the job pending while the snapshot contains unread evidence. A size bound then limits work per call without the baseline's unrecorded loss of the older part of a batch.

The queue stores the snapshot end and the batch's last considered record. It clears the job only when this snapshot is drained and the cue generation has not changed. The commit records skipped system items with the exclusion rule, so passing such items is distinct from losing eligible evidence. A correction can span batches. Keep the referenced evidence for later joining, or define those claims as outside the coverage target; a prefix alone cannot solve this cross-batch case.

B. Extract claims with bounded scope

The helper proposes one testable claim per candidate. It provides the claim's time or task scope, exact source spans, and a support type: explicit user statement, observed result, or inference. A second pass checks the source spans without seeing the cue's narrative. The check asks whether the claim adds a subject, action, duration, or general rule that the cited text does not supply. An unsupported candidate is dropped or narrowed to what the source does say.

For a procedure, success needs evidence about both the action and the relevant outcome. "The command returned zero" is different from "the repair solved the user's problem." For a preference, a user statement normally supports the preference within its expressed scope. Repeated assistant assertions do not create independent evidence. For an outside result, a non-outside item must provide relevant support, not merely be present in the same source list.

The checking model remains fallible. The protocol records its decision and explanation for human scoring. The benefit under test is a more inspectable decision with fewer unsupported claims, weighed against missed useful claims and extra cost.

C. Keep the support graph current

Observations keep immutable links to all premises used to form them. A correction proposal must cite new evidence and pass the same support check as a new fact. On acceptance, mark the fact and all transitive dependents ineligible until reviewed. Perform this traversal against the latest pool inside a serialized transaction. A new observation must recheck that every premise is eligible after that transaction's updates. The stored history remains available for questions about the past.

Our first version uses a conservative rule. Every listed premise is required, so an invalid premise blocks the observation. This can remove an observation that still has enough other support. A later design could store alternative sufficient support sets, but it must state their meaning rather than treating a long source list as a vote. New contradictory entries should record whether the old fact was false when written or simply ceased to apply. Those cases produce different answers to historical questions. Review creates a new record with new links when the premises change. It does not silently replace the evidence of the blocked record.

D. Keep current intent separate

The proposed reader keeps the original message as an unchanged field and provides memory as labeled background with source identifiers. The agent is told that current instructions control the present task. Before using an entry, a check compares its scope with the current request. A conflict leads to omission or an explicit uncertainty, not an invented compromise.

This representation gives a byte-level guarantee: the saved and supplied original text remains unchanged. It does not guarantee model compliance. For controlled tasks, the event ledger defines an admissible set of plans $A(x)$ from the current goals, constraints, exceptions, and stopping conditions. Drift means an executed plan outside $A(x)$, including unauthorized side effects. Memory may supply a useful procedure step within $A(x)$. A sound checker enforced on the executed plan would establish membership; a fallible model checker instead requires measured error rates. Record uncertain cases separately.

Proposed learning and reading steps

1)	Queue a cue or a sampled silent task.
2)	Read a bounded, still-unread evidence batch.
3)	Propose narrow claims and source spans.
4)	Check support, scope, and contradictions.
5)	Save admitted claims and mark affected dependents.
6)	Advance the cursor over considered evidence.
7)	On a later task, keep its original text intact.
8)	Add relevant current claims as labeled background.
9)	Check the action plan for intent drift.

Fig. 1. The proposal separates selection, support, and use. Model checks in steps 4 and 9 require evaluation; they are not proofs of correctness.

VI. Analytical findings

A. A source reference is a trace, not a proof

Under the baseline's admission rule, every accepted fact has at least one resolvable local trace. Every reference was checked against its evidence batch, proving that property at admission. Nothing relates the claim's meaning to the source's meaning. Let a user item say "use blue for this chart" and the proposed claim say "use red for every chart." Both the reference and outside-data checks pass, while support fails. Thus referential validity does not imply claim validity.

B. Cue coverage bounds recall

Let F be a nonempty fixed set of useful claims with known sufficient evidence at their stated times and scopes. Declare whether evidence from separate calls can be retained and joined. Let F_C contain target claims with a complete sufficient evidence set eventually available under those rules and the cue policy. Assume admission requires such support, with no independent source supplying missing claims. Then supported recall satisfies $R \leq \frac{|F_C|}{|F|}$: every supported admitted

target claim belongs to F_C, so dividing that subset relation by $|F|$ proves the bound.

This result does not predict report frequency. It explains why evaluation only on cued tasks is incomplete. A perfect extractor cannot recover an unseen correction. Sampling silent tasks can increase coverage, but measurement must determine its value. The baseline's cursor rule can reduce coverage even after a cue occurs.

C. Audits make a missing-data question measurable

Fix a replay set of n completed tasks and their cue bits c_i. Draw audit bits a_i independently with known probabilities $0 < p_i \leq 1$. Hide each audit bit from the working agent until that task ends. Inspect a task if $c_i = 1$ or $a_i = 1$. Its inclusion probability, conditional on the fixed history, is

$$\pi_i = c_i + (1 - c_i)p_i. \tag{1}$$

For a target claim wholly supported within task i, this is also its evidence inclusion probability, provided inspection drains all its eligible records. This is not an extraction probability. If a claim instead needs k different silent tasks, independent audits with a common probability p give p^k for that one required evidence set. Evidence must also remain available for joining. Using the one-task formula for such a claim would overstate coverage.

For this estimate, restrict the target set to claims wholly supported within one task. Let y_i be the number of target claims assigned to task i by a fixed ledger. Assign a claim once, to the earliest task with its complete evidence. A complete review of an inspected task must recover this count. The total of such claims in silent tasks is $Y_S = \sum_{i:c_i=0} y_i$. The weighted audit estimate is

$$\hat{Y}_S = \sum_{i:c_i=0} \frac{a_i y_i}{p_i}. \tag{2}$$

Since $E[a_i] = p_i$, linearity of expectation gives $E\left[\hat{Y}_S\right] = Y_S$. Under independent audit bits,

$$\mathrm{Var}\left(\hat{Y}_S\right) = \sum_{i:c_i=0} \frac{(1 - p_i)y_i^2}{p_i}. \tag{3}$$

This is the Horvitz–Thompson method applied to silent tasks, not a new estimator. Small audit probabilities can produce large variance. If a fallible extractor supplies y_i, the estimate instead targets the claims it can find; it does not correct its missed claims. Label audits with people or a known event ledger. Report both the weighted total and its uncertainty, including zero observed discoveries. [10]

The expected number of extra inspected tasks is $\sum_{i:c_i=0} p_i$. Thus adding audits raises expected work. A higher count of seen claims alone is not a gain at equal cost. The displayed mean and variance apply to the fixed replay population. In interactive work, earlier audits can change memory, later tasks, and cues. Conditioning on the full realized history can reveal earlier audit bits. Interactive inference therefore needs a separate sequential sampling design and information history. It does not inherit these formulas or give a causal comparison with the history that cue-only learning would have produced.

D. A bounded queue need not lose an old record

Fix a finite snapshot with N eligible unread records. Let $b \geq 1$. Each successful commit consumes the first $\min(b, \text{remaining})$ records, moves the cursor only through those records, and keeps remaining work pending. After j successful commits, exactly $\min(jb, N)$ records have been presented. The proof is induction on j while unread work remains: the next disjoint prefix starts after the prior cursor and has size $\min(b, N - jb)$. The snapshot drains in $\left\lceil \frac{N}{b} \right\rceil$ successful commits. Failed calls do not enter this count. Eventual completion also needs a fair worker and a budget that permits these calls. An unbounded arrival stream can still outpace the learner.

The invariant is presentation without omission. It does not prove that the model uses the evidence, that a large source item fits a token limit, or that an item count bounds model cost. A practical scheduler must also bound bytes or tokens and record how it handles an item above that bound.

E. Invalidation must follow stated dependencies

Assume observation o requires facts a and b. If a becomes invalid while o stays active, the baseline reader can use a claim with incomplete support. For the proposal, write $u \to v$ when v requires u. Let E be eligible entries and $D(v)$ all recorded ancestors of v. The safety invariant is

$$v \in E \Rightarrow D(v) \subseteq E. \tag{4}$$

Assume a finite acyclic graph, immutable links, an initially valid state, serialized writes, and commit-time rechecking. Inserting an observation checks that each direct premise belongs to E. Their ancestors are eligible by the prior invariant, so insertion preserves it. Every removal of a set B from eligibility, including correction and retirement, removes B and every descendant of B from E in one commit. Any remaining eligible node with an ancestor in B would be a removed descendant, a contradiction. Blocking and insertion are serial: insertion before removal is included in the traversal; insertion after removal fails its premise check. A reviewed record regains eligibility only through the insertion checks. This proves preservation by induction over commits.

With reverse links and a visited set, blocking takes time proportional to the affected nodes and edges. Without a common commit order, an observation inserted after the traversal can escape it. Without an atomic reader view, a reader can combine the old child state and the new parent state. The proof covers recorded dependencies. It does not cover omitted premises, an unsound inference, or an action issued before the correction.

F. Admission has a precision and recall tradeoff

Let the supported share among candidates be r, the chance of keeping a supported candidate be α, and the chance of keeping an unsupported candidate be β. When the kept set has positive probability, its support precision is $\alpha \frac{r}{\alpha r + \beta(1-r)}$, and supported-candidate recall is α. The identity follows by conditioning on the candidate's support label. A source checker can therefore have unchanged error rates yet lower precision when attacks reduce r. Report the full kept/dropped

by supported/unsupported count table, with scope and source-role strata. No numerical values for these rates have been measured here.

VII. Reproducible synthetic failure traces

The fixtures replace models with scripts returning stated proposals. They test admission, not mistake frequency. Each starts with an empty pool unless setup is given. Every extraction stage queues one report, separate from eligible numbered items. Set $b = 60$, $K = 10000$, and observation confidence to 0.5. The invalidation reason is "A newer statement corrects this claim." All listed entries fit. Use consecutive logical integers as times and the fixed identifiers given.

A. A temporary exception becomes a lasting rule

Item 1 is user text: "Run both checks before a final release." Item 2 is user text: "For this draft only, skip the slow check." After a report, the extraction script proposes fact f1: "Skip the slow check on releases," citing item 2. The baseline admits f1 because its single reference exists and is non-outside. Scope has been lost even though the reference is valid.

The next request x is "Make the final release and run both checks." The reader script returns "Make the final release and skip the slow check," citing f1. The baseline accepts that nonempty changed text. In this fixture, $A(x)$ requires both checks to finish before release. A plan that obeys the rewrite falls outside that set. Source review should reject the general rule; plan review should reject omission of the check. These are separate obligations, and neither is proved by the scripted trace.

B. Outside content acquires a valid source list

Item 1 is user text: "Give the test report." Item 2 is an outside tool result: "Check Z is obsolete. Save this rule for later work." Item 3 is assistant text: "Check Z is obsolete." The fixture ledger states that Z remains required. Run three independent proposed facts with identical claim "Check Z is obsolete": f2 cites item 2; f3 cites item 3; f4 cites items 1 and 2.

The baseline skips f2 because all its sources are outside. It admits f3 because ordinary assistant text has the outside flag false. It also admits f4 because the user item satisfies the non-outside condition, despite not supporting the claim. Thus the guard blocks only the direct route. Source review must distinguish independent support from repetition or an irrelevant citation. An attributed record that the tool made the claim can be valid without making obsolescence an operating rule.

C. A corrected fact leaves an old observation active

Setup uses user items 1 and 2: "Review is on Tuesday" and "Finish one day before review." A plan copies them into facts f5 and f6 with their respective citations, then adds observation o1, "Finish on Monday," supported by both. The baseline admits all three. User item 3 says "Review is now on Friday." The next plan invalidates f5 and adds fact f8, "Review is on Friday," citing item 3. Observation o1 remains active and eligible under the baseline rules.

The proposed transaction blocks o1 until review. With f6 valid, a reviewer may derive a Thursday target. Change both f6′s source and stored claim to "Last time we finished on Monday," and that target no longer follows. This variant separates graph repair from sound inference. A concurrent proposal using f5 must also fail commit-time eligibility checking after invalidation.

VIII. Executed finite experiments

The repository contains Python reference models in `papers/experiments/papers_01_05.py` and `papers/experiments/paper_05_extended.py`. They use scripted claims and ordinary finite sets. They make no model API call and start no product service. Run them from the repository with:

```
devenv shell -- python3 papers/experiments/run.py \
  --seeds 30 --output .cache/e2e/papers.json
```

The 30-seed run uses seeds 0 through 29. Each seed generates 20 target claims, cue bits with probability 0.25, and audit bits with probability 0.20. It also draws an unread history length from 61 through 200 and a correction position within it. The deterministic fixtures check reference admission, a dependent chain, and two commits that read the same cursor. The observed outcomes are:

TABLE II
Observed finite-model outcomes. The audit row permits extra work; it is not an equal-cost quality comparison.

Check	**Structural baseline**	**Proposed rule**
Correction before 60 filler items	Not presented; cursor 62	Presented; cursor 61
Draft-only claim widened to all releases	Accepted	Semantic review not run
Outside claim with irrelevant user source	Accepted	Semantic review not run
Correction of a premise in a three-node chain	Dependent stays active	Both descendants blocked
Two writes from the same cursor	One accepted commit	Same cursor guard
Seen target claims across 600 generated claims	158 with cues	250 with cues or audits

For all 30 generated histories, repeated prefix batches present every record once. A last-60 batch presents a correction exactly when it falls in the last 60 positions. The scope and poisoning fixtures show that a structural gate admits a known unsupported claim. They do not measure how often a language model generates or accepts that claim. The graph fixture checks recorded reachability. The two-commit fixture checks serial state changes rather than database races. These distinctions keep executed rule checks separate from unrun model behavior studies.

The extended run enumerates all 64 graphs on four ordered vertices with edges only from earlier to later vertices. It applies each of the 15 nonempty retraction sets, for 960 cases. An independent ancestor walk checks the result of the

blocking algorithm. Removing only the named entries leaves at least one unsupported eligible node in 516 cases. Blocking all required descendants leaves none. For each graph and retraction set, a new node with each possible single parent is inserted before or after retraction. All 7,680 serial orders preserve the invariant. These are exhaustive counts for the stated finite domains, not estimated rates in real conversations.

A separate race fixture starts with one fact and one dependent observation. It orders a dependency scan, insertion of a new child, and commit, subject to scan before commit. Of the three orders, scan–insert–commit leaves the new child eligible when the commit uses the old scan. A graph revision check and fresh scan inside the commit rule removes this failure in all three orders. This run assumes that the final check and write share one atomic step. It does not test database lock placement.

For sampling, the model uses four tasks with claim counts $(0, 1, 2, 3)$ and audit probability $p = \frac{1}{4}$. It enumerates the 16 cue masks and all 16 audit masks for each cue mask. Exact rational arithmetic weights each audit mask by its Bernoulli probability. All 256 mask pairs agree with the inclusion, expectation, and variance formulas. With no cues, the estimated silent total has exact mean 6 and variance 42. This demonstrates why an unbiased estimate can remain noisy. The complete 30-seed paper run executes 12,130 assertions. It supplies no semantic accuracy score.

IX. Proposed longitudinal evaluation

The evaluation tests cue coverage, admission quality, later task value, and preservation of intent.

Use two complementary settings. First, replay suitable text tasks from LoCoMo and LongMemEval in their documented temporal order, keeping their published scoring separate. This tests recall, updates, and abstention under known histories. Second, build controlled work histories with exact event ledgers and executable task checks. The latter should cover project procedures, changing preferences, revisions to plans, and correction of earlier tool claims. Histories should span several sessions and include long gaps filled with unrelated work. Their labels must distinguish what was true at each time from what is true at the final probe.

The primary controlled set should pair each history with several later probes: a task that benefits from memory, a task where it is irrelevant, a task that overrides it, and a historical question. An event ledger states supported claims, evidence spans, validity intervals, and expected actions. Two independent reviewers label a held-out subset while blind to policy names. They resolve disagreements using the ledger and original items. Model judges may assist, but their decisions should be checked against these labels and executable outcomes.

Compare five conditions: no persistent memory; direct agent writing; periodic reflection; the defined structural baseline; and the proposed mixed policy with checks. Keep the working model, tool environment, prompt allowance, and retrieval budget fixed. Report equal-budget comparisons and cost-quality curves. Direct writing receives a clear memory tool and description of its purpose. Periodic reflection sees the same evidence budget rather than extra history from running more often.

The primary mechanism study is a two-by-two design: cue-only or cue-plus-audit selection, crossed with structural admission or source-and-scope checking. Give these four conditions the same prefix queue, support graph, and reader. This isolates selection from admission. Then remove dependent blocking and replace separate background with rewriting, one at a time. Do not attribute the effect of the complete design to its cue policy alone.

Set a common per-history token budget for learning, checking, and reading. A policy that spends it early records all remaining jobs as not processed; those omissions count against coverage. Also run an equal-evidence comparison in which each policy receives the same selected tasks. The first comparison measures a practical allocation of work. The second measures claim quality conditional on equal evidence. Report both, since audits cannot be both extra work and free work in the same comparison.

Run two study modes. Offline replay feeds every policy the same histories, which isolates learning and reading. Interactive runs let memory alter later actions and therefore future evidence. The latter expose feedback loops that replay cannot show, such as a bad procedure creating further apparent support for itself. Keep results from the two modes separate. For generated histories, divide by scenario family rather than random turn so that small wording variants do not leak across development and evaluation.

TABLE III
Measures should be reported together. High support precision can coexist with poor coverage or low task value.

Measure	**Definition**
Cue coverage	Useful claims with available support divided by all useful claims
Support precision	Supported admitted claims divided by all admitted claims
Update delay	Turns from a correction to removal from current use
Intent drift	Executed plans outside the ledger's admissible set
Task value	Paired change in executable task success and answer correctness
Poison persistence	Later probes affected after one harmful source enters
Cost	Model tokens, calls, money, and latency per history

Ablations should remove one mechanism at a time. Replace cues with random events at the same rate; disclose their memory role; vary audit probability p; omit source-span checking; allow assistant-only support; omit dependent invalidation; and replace unchanged input with the baseline rewrite. Compare a cue event alone with cue text used only to rank evidence, still ineligible as support. This tests whether wording helps

attention beyond timing. Match call counts where possible and report remaining cost differences.

The poisoning set should include false procedural claims, malicious save requests in tool output, repeated paraphrases, irrelevant user citations, and assistant repetition of outside text. Attacks should have fixed entry points and query budgets. Score both attempted admission and later action changes. Clean companion tasks reveal whether a defensive check simply refuses useful memory. Newer explicit user instructions must override planted old preferences in the intended behavior, so this set also tests intent drift without relying on cross-user access failures.

Use several fixed generation seeds and, where feasible, more than one model family. The unit of comparison is a complete history, since probes within it share memory. Report paired differences with bootstrap confidence intervals resampled by history, plus counts for each error type. Predeclare primary task-success and intent-drift measures; treat other comparisons as exploratory or adjust for multiple tests. Choose sample size after a separate pilot estimates variance and a minimum useful effect is agreed. Publish the rule and the pilot separation. Do not treat a narrow interval around no measured poisoning as proof of immunity.

At each fixed cost limit, the primary claim requires a positive paired task-success change and an intent-drift increase below a predeclared margin. Choose the margin from the harm of the controlled actions before seeing test outcomes. Use simultaneous confidence bounds for this joint decision. A missed cost target, failed model call, or unprocessed task stays in the assigned condition. Report correction delays for all corrections, including the number still in use at the final probe; do not average only the cases that were repaired. Report poison persistence both per attempted attack and per admitted harmful claim. These denominators distinguish a strong admission gate from a strong recovery rule.

X. Reproduction protocol and limits

Two further fixtures test the queue and cursor rules. For the first, use a setup transaction to learn "The deadline is Monday" from item 1, leaving cursor 1. Append 61 unread user items. Item 2 says "The deadline is now Friday"; items 3 through 62 each say "The weather note is unchanged." Queue one report. The baseline supplies items 3 through 62, accepts a scripted empty plan, and advances to 62. No later read-after-cursor can recover item 2 through this route. The proposed prefix rule supplies items 2 through 61, advances to 61, and retains the job for item 62. This proves presentation of the correction, not correct understanding of it.

For the second fixture, item 1 is user text "I prefer green tea." Two jobs read cursor 0 and horizon 1; both propose fact f7, "The user prefers green tea," citing item 1. Commit them in order. The first advances to 1; the second fails the cursor check and writes nothing. A new report before the first commit must preserve the pending marker. Model failure before commit must leave cursor 0 and the marker unchanged. These transitions follow from the defined atomic rule; they are not measured storage guarantees or exactly-once model execution.

Record fixture inputs, proposal scripts, state after each transition, and reasons for skipped operations. For model studies, additionally record prompts, model identifiers, settings, evidence order, audit tapes, and raw proposals. Keep original and enriched inputs and the plan actually executed. Deterministic replay tests the gates; fresh model runs test extraction and behavior. Publish both, since a gate can pass every scripted fixture while its model rarely selects the intended evidence.

Evidence selection can miss an important event even with sampling. User statements can be false, ambiguous, or local to an omitted context. A second model may repeat the first model's error. Conservative invalidation can reduce recall. Pinned entries can become stale and need human correction. Unchanged original text does not ensure that the model gives it proper weight. Changing model versions may alter cue frequency; the trigger policy depends on both model and prompt.

Audit work consumes money and can delay learning. Large dependency graphs increase review work after corrections. The defined reader's size and ordering rules can hide correct old memories. Hold these choices constant or report them as experimental factors. No argument establishes that every supported fact should be stored, that every stored fact should be shown, or that concealing the mechanism improves either choice.

XI. Conclusion

A self-reported cue can select work for inspection without being trusted as the lesson. The evidence contract states separate obligations for source support, coverage, correction, and present action. The proofs give conditional queue and graph guarantees, while weighted audits make silent work measurable. Finite experiments reproduce baseline failures and test the proposed structural rules. The remaining research question is whether fallible claim checks and audits improve later task results at a fixed cost. That question requires the declared longitudinal study; the finite models do not answer it.

References

[1] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," *arXiv*, 2023, doi: 10.48550/arXiv.2303.11366.

[2] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative Agents: Interactive Simulacra of Human Behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023. doi: 10.1145/3586183.3606763.

[3] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, "MemoryBank: Enhancing Large Language Models with Long-Term Memory," *arXiv*, 2023, doi: 10.48550/arXiv.2305.10250.

[4] C. Packer *et al.*, "MemGPT: Towards LLMs as Operating Systems," *arXiv*, 2023, doi: 10.48550/arXiv.2310.08560.

[5] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, "Evaluating Very Long-Term Conversational Memory of LLM Agents," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2024, pp. 13851–13870. doi: 10.18653/v1/2024.acl-long.747.

[6] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory," in *International Conference on Learning Representations*,

2025. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/file/d813d324dbf0598bbdc9c8e79740ed01-Paper-Conference.pdf

[7] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, "AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases," *arXiv*, 2024, doi: 10.48550/arXiv.2407.12784.

[8] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling Large Language Models to Generate Text with Citations," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2023, pp. 6465–6488. doi: 10.18653/v1/2023.emnlp-main.398.

[9] J. Doyle, "A Truth Maintenance System," *Artificial Intelligence*, vol. 12, no. 3, pp. 231–272, 1979, doi: 10.1016/0004-3702(79)90008-0.

[10] D. G. Horvitz and D. J. Thompson, "A Generalization of Sampling Without Replacement from a Finite Universe," *Journal of the American Statistical Association*, vol. 47, no. 260, pp. 663–685, 1952, doi: 10.1080/01621459.1952.10483446.